Parte IV. Informe sobre su proceso de solución

Integración por Monte Carlo en memoria compartida con OpenMP

Nancy Liliana Saenz Moreno · Código 2224510 · Semilla 2224510 · UIS, grupo 28661

Redacte un informe breve, de una a dos páginas, en el que usted explique el trabajo que realizó para llegar a su solución. No se trata de repetir la teoría: se califica la claridad con la que usted da cuenta de su propio proceso. Su informe debe describir, como mínimo:

1. **las decisiones de diseño que tomó y por qué las tomó: estructura del programa, cláusulas de OpenMP empleadas y manejo de las semillas de los generadores de cada hilo;**
2. **los problemas que aparecieron durante el desarrollo y cómo los diagnosticó y corrigió (por ejemplo: resultados distintos en cada ejecución, estimaciones sesgadas, condiciones de carrera) y qué evidencia le permitió identificar cada uno;**
3. **el método que usó para validar la correctitud de sus resultados, tanto la estimación de π como la reproducibilidad con distinto número de hilos;**
4. **la conexión explícita entre su informe y sus datos: cite valores concretos de su tabla (por ejemplo, por qué su mediana con $p = 4$ resultó como resultó) en lugar de descripciones genéricas que podrían aplicarse a cualquier entrega;**
5. **una reflexión final sobre lo que aprendió y sobre lo que haría de manera diferente en un próximo trabajo paralelo.**

1. Decisiones de diseño. Quise cambiar lo menos posible el programa del enunciado: el mismo ciclo, el mismo `rand_r` y la misma prueba $x^2 + y^2 \leq 1$. Lo nuevo es que los 10^8 puntos se dividen en 1.000 bloques de 100.000 y cada vuelta del `omp parallel for` hace un bloque. Usé `reduction(+:dentro)` para que cada hilo cuente en su copia y se sume una vez al final (más barato que `critical` o `atomic`), `schedule(static)` porque todos los bloques cuestan lo mismo, y el número de hilos como argumento o con `OMP_NUM_THREADS`. Para las semillas, como el resultado debe ser igual con cualquier p, hice que los números dependan **del bloque y no del hilo**: `saltar` lleva cada bloque a su lugar en la lista de `rand_r`, y así cada bloque usa siempre los mismos números, sin importar qué hilo lo ejecute.

2. Problemas, diagnóstico y evidencia.

Problema	Cómo lo noté (evidencia)	Qué hice
Semilla equivocada	Usé la del enunciado (20261018); el profesor aclaró que es el código de cada uno	La cambié a 2224510 y repetí todo
Generador compartido (carrera)	Con `rand()` y 8 hilos el conteo cambió en cada corrida y tardó 11 s contra 1,6 s	Cada bloque con su propio estado
Misma semilla en los hilos	Con 8 hilos $\pi = 3{,}142351$, igual al de solo $N/8$ puntos	Cada bloque empieza en otro lugar (`saltar`)
Contador sin `reduction`	Con 1 hilo daba bien; con 8, π = 0,654; 0,694 y 0,640	`reduction(+:dentro)`
Semillas "distintas" que no lo eran	En la Parte I, las semillas $2224510 + 1000k$ daban un error medio 2,7 veces menor que el teórico: sus listas se solapaban	Tramos sin solapamiento con `saltar`; el error coincidió con la fórmula
Mediciones con ruido	Una corrida de 12,4 s (lo normal era 1,25 s); una lectura de frecuencia de 544 MHz con el procesador ocupado	Mediana y muestras de frecuencia más largas

3. Validación. Con $N = 10^8$ obtuve $\pi = 3{,}141668$, con un error (0,0000756) menor que el típico esperado (0,000164). Para la reproducibilidad exigí que el **conteo fuera idéntico**, no solo parecido: **78.541.707** en el secuencial, con 1, 2, 4 y 8 hilos, con `OMP_NUM_THREADS` y en las 35 corridas de la Tabla de la Parte III. Además verifiqué que el estado inicial de cada hilo con `saltar` coincide con el de `rand_r` avanzando uno por uno, y `montecarlo.c` compila sin advertencias con `-Wall -Wextra`.

4. Conexión con mis datos. Con $p = 4$ los siete tiempos fueron de 0,3449 a 0,3879 s y la **mediana quedó en 0,3700 s**, por eso $\boldsymbol{S(4) = 1{,}2549/0{,}3700 = 3{,}39}$ y no 4. Crear los hilos cuesta solo 0,57 ms (0,15 %) y la frecuencia bajó apenas un 3 %; probé `schedule(dynamic)` por si unos hilos esperaban a otros y no mejoró (0,456 contra 0,412 s), así que el resto lo atribuyo a que Windows y WSL usan los mismos núcleos. Con $p = 8$, $S(8) = 5{,}02$, por debajo de Amdahl con $f = 0{,}940$ (5,64): ahí sí hay causas medidas: crear 8 hilos costó 13,6 ms (5,4 %), la frecuencia bajó un 15 % y hay 2 hilos por núcleo. La mediana también fue clave: la corrida de 12,41 s habría hecho que el promedio diera un $S(4)$ imposible de 7,7.

5. Reflexión. Aprendí que la reproducibilidad hay que diseñarla desde el principio: atar los números aleatorios al bloque y no al hilo la dio para cualquier p. También que un error de concurrencia puede esconderse (sin `reduction` acierta con un hilo), que dos semillas distintas no garantizan números distintos, y que medir en un portátil tiene ruido: hay que repetir y usar la mediana. En un próximo trabajo confirmaría primero los datos asignados (la semilla), usaría un generador hecho para trabajar en paralelo y sin el límite de 7×10^8 puntos, y fijaría cada hilo a un núcleo (`OMP_PROC_BIND`) para separar mejor el efecto del hiperprocesamiento.

Anexo. Bitácora de IA

Herramienta	Propósito	Qué aportó	Qué acepté o descarté y por qué
Claude Code (27/10/2026)	Responder la Parte I	Programas de apoyo y figuras	Acepté apoyar cada respuesta con un experimento real; descarté una versión con semillas correlacionadas
Claude Code (27/10/2026)	Programar la Parte II	Programa paralelo y programas de prueba de cada error	Acepté el diseño por bloques; cambié la semilla del enunciado por mi código
Claude Code (27/10/2026)	Medir la Parte III	Scripts de medición y gráficas	Acepté medir cada factor; descarté una lectura de frecuencia errónea
Claude Code (27/10/2026)	Redactar la Parte IV	Borrador del informe	Acepté la estructura; revisé que cada cifra coincida con mis salidas